

radcoolpv

Physics, Numerical Models, YAML Cases, and Validation

Project manual

30 July 2026

Contents

1 Purpose and scientific status	1
2 Installation	2
3 Optical model	2
3.1 The S4 problem	2
3.2 Directions and polarization	3
3.3 Emittance and Kirchhoff's law	3
3.4 Materials and numerical validity	3
4 Atmosphere, thermal balance, and PV model	4
4.1 Atmospheric and thermal radiation	4
4.2 Solar absorption and steady state	4
4.3 I–V calculation	5
4.4 Temperature reduction	5
5 YAML examples	5
5.1 One file with multiple cases	5
5.2 Normal and directional emittance	6
5.3 Hemispherical emittance	6
5.4 Layer stack and materials	7
5.5 Thermal case and temperature reduction	7
6 Outputs and reproducibility	7
7 Validation evidence	8
8 Required verification for new cases	8
References	9

1 Purpose and scientific status

radcoolpv calculates optical properties of periodic photonic structures for radiative cooling and couples them to a silicon PV thermal/electrical model. The active implementation is based on

the MATLAB+Lua/S4 `matlab-radCoolPV` model. Live optics use only the Stanford S4 Python extension. S4 is imported lazily; reduced-spectrum and free-form readers can run without it.

The package reports spectral reflectance R , transmittance T , total absorptance A , silicon-layer absorptance A_{Si} , and, under the conditions stated in Section 3.3, emittance. The thermal stage reports operating temperature, I–V characteristics, maximum-power-point output, efficiency, and a temperature reduction relative to a user-specified reference.

The library is not a universal PV module model. Its central assumptions are a periodic coherent optical unit cell, scalar material permittivities, a lumped steady-state thermal balance, and the silicon diode model inherited from the MATLAB code. Validation status is evidence-labelled in Section 7; a passing software test is not treated as proof of physical validity.

2 Installation

```
./install.sh
source .venv/bin/activate
python -c "import radcoolpv; print(radcoolpv.__version__)"
```

The thermal/PV stage and spectrum readers use the Python dependencies installed by `install.sh`. Live optics requires a separately compiled S4 Python module:

```
brew install fftw suite-sparse openblas lapack boost
git clone https://github.com/phoebe-p/S4
cd S4
make -f Makefile.m1 S4_pyext
python -c "import S4; print(S4.__file__)"
```

The S4 import occurs only when a case with `geometry.source: s4` executes.

3 Optical model

3.1 The S4 problem

The YAML structure is ordered from the incident medium toward one semi-infinite terminal layer. S4 expands the in-plane fields in a Fourier basis of size `simulation.s4_modes`. Flat, circular, rectangular, and polygonal regions are supplied to S4 analytically; spheres and hemispheres use the configured stack of circular slices.

At each wavelength, direction, and polarization, the code evaluates the integrated normal Poynting flux. With incident flux P_{inc} ,

$$R = -\frac{P_{\text{top}}^-}{P_{\text{inc}}}, \quad (1)$$

$$T = \frac{P_{\text{bottom}}^+}{P_{\text{inc}}}, \quad (2)$$

$$A = 1 - R - T. \quad (3)$$

Unlike the old Lua output, T is normalized by incident flux at oblique incidence. The silicon-layer absorptance uses the net flux at the two interfaces of the silicon layer itself:

$$A_{\text{Si}} = \frac{P_{\text{net}}(z_{\text{Si,top}}) - P_{\text{net}}(z_{\text{Si,bottom}})}{P_{\text{inc}}}. \quad (4)$$

This prevents absorption in a layer below silicon from being assigned to the PV cell.

3.2 Directions and polarization

One YAML case supports one of three modes:

- **normal**: $\theta = 0$;
- **specific**: one user-defined polar angle θ and azimuth ϕ ;
- **hemispherical**: a two-dimensional (θ, ϕ) quadrature.

polarization is TE, TM, or unpolarized. TE and TM are never assumed equal solely because $\theta = 0$; anisotropic unit cells such as one-dimensional gratings can give different normal-incidence responses. Unpolarized output is

$$X_{\text{unpol}} = \frac{X_{\text{TE}} + X_{\text{TM}}}{2}, \quad X \in \{R, T, A, A_{\text{Si}}\}. \quad (5)$$

For one polarization p , the hemispherical property is

$$X_{\text{h},p}(\lambda) = \frac{1}{\pi} \int_0^{2\pi} \int_0^{\pi/2} X_p(\lambda, \theta, \phi) \cos \theta \sin \theta \, d\theta \, d\phi. \quad (6)$$

The implementation applies Gauss–Legendre quadrature in $u = \sin^2 \theta$ and a uniform periodic azimuth rule. It avoids the grazing endpoint and includes a separate zero-weight normal probe for the PV luminescence term. Quadrature weights sum to one. A true hemispherical result therefore requires convergence with respect to both **hemisphere_theta_points** and **hemisphere_azimuth_points**.

3.3 Emittance and Kirchhoff’s law

The code labels total absorptance as emittance using directional Kirchhoff reciprocity,

$$\epsilon_p(\lambda, \theta, \phi, T) = A_p(\lambda, \theta, \phi, T). \quad (7)$$

This identification requires a passive reciprocal structure in local thermal equilibrium, evaluated with the same wavelength, direction, polarization, and temperature-dependent material state. It is not automatically valid for nonreciprocal, active, nonlinear, or nonequilibrium emitters. The present material models are temperature-independent.

3.4 Materials and numerical validity

Tabulated models interpolate n and k linearly and use $\epsilon = (n + ik)^2$. Requests outside a table’s wavelength interval raise an error; endpoint clamping and silent extrapolation are forbidden. Validation E uses the unmodified refractiveindex.info Olmon evaporated-Au record. Its SiO₂ and Si refractive-index tables are inherited from **matlab-radCoolPV**: the former is labelled Palik/Kitamura; the latter has no bibliographic source and is made lossless to implement the paper’s explicit nonabsorbing-Si assumption. The current refractiveindex.info database has no records labelled with the paper-cited Palik/Kitamura Si or SiO₂ sources. This provenance gap is retained as a limitation rather than silently replacing the models.

Every scientific result must demonstrate:

1. passivity and finite R, T, A, A_{Si} , with $R + T + A \approx 1$;
2. convergence in S4 Fourier basis size;
3. for hemispherical results, convergence in both angular counts.

4 Atmosphere, thermal balance, and PV model

4.1 Atmospheric and thermal radiation

For zenith transmittance $\tau_{\text{atm}}(\lambda)$, the directional atmospheric emittance is

$$\epsilon_{\text{atm}}(\lambda, \theta) = 1 - \tau_{\text{atm}}(\lambda)^{1/\cos\theta}. \quad (8)$$

The surface-atmosphere product is integrated with the same angular quadrature as the surface emittance. With Planck spectral radiance $B_{\lambda}(T)$,

$$P_{\text{rad}}(T) = \pi \int \epsilon_{\text{h}}(\lambda) B_{\lambda}(T) d\lambda, \quad (9)$$

$$P_{\text{atm}}(T_{\text{amb}}) = \pi \int \langle \epsilon \epsilon_{\text{atm}} \rangle_{\text{h}} B_{\lambda}(T_{\text{amb}}) d\lambda, \quad (10)$$

$$P_{\text{conv}}(T) = h (T - T_{\text{amb}}). \quad (11)$$

The convection coefficient h is the total effective nonradiative coefficient for the reference area used by the balance. It is an external lumped-model input and is not deduced from geometry or weather data. A per-surface coefficient must be combined explicitly by the user; the library does not multiply h by an assumed number of exposed surfaces.

4.2 Solar absorption and steady state

Total absorbed solar power is

$$P_{\text{sun}} = \int A(\lambda) I_{\text{AM1.5G}}(\lambda) d\lambda. \quad (12)$$

For a literature cooling curve that prescribes absorbed solar power directly, the `absorbed_solar_power` key in the `thermal` section overrides this integral. Validation E uses the paper value 808 W m^{-2} .

The cooling-power convention is

$$P_{\text{cool}}(T) = P_{\text{rad}} - P_{\text{atm}} + P_{\text{conv}} - P_{\text{sun}} + P_{\text{MPP}} + P_{\text{nt}}, \quad (13)$$

where P_{MPP} is electrical power removed from the cell and P_{nt} is the non-thermal luminescence term. Equilibrium satisfies $P_{\text{cool}}(T_{\text{eq}}) = 0$.

Cooling-curve cases specify their temperature interval in YAML. Standard PV cases evaluate T_{amb} through $T_{\text{amb}} + 150 \text{ K}$. If the balance does not bracket a root, the run raises an error. It never reports a sweep endpoint as an equilibrium temperature.

4.3 I–V calculation

The short-circuit current uses IQE, silicon-layer absorptance, and the solar photon flux up to the silicon bandgap:

$$J_{\text{sc}} = q \int_0^{\lambda_g} \text{IQE}(\lambda) A_{\text{Si}}(\lambda) \Phi_{\text{sun}}(\lambda) d\lambda. \quad (14)$$

At each T, V , the current density J solves

$$0 = -J + \frac{V - JR_s}{R_{\text{sh}}} + J_0(T) \left[\exp\left(\frac{q(V - JR_s)}{k_B T}\right) - 1 \right] + J_{\text{Auger}}(T, V) - J_{\text{sc}}. \quad (15)$$

Output power is $-JV$. A three-point parabolic refinement gives the MPP between voltage samples. Open-circuit voltage and operating temperature must be bracketed; otherwise the calculation fails explicitly.

The bandgap calculation preserves the MATLAB scalar-reduction convention for parity. This is a model-compatibility choice and should not be confused with a fully temperature-resolved semiconductor bandgap model.

4.4 Temperature reduction

When `thermal.reference_temperature` is supplied,

$$\Delta T_{\text{reduction}} = T_{\text{reference}} - T_{\text{eq}}. \quad (16)$$

The user must define the reference physically, for example the operating temperature of a bare module under the same boundary conditions. The library does not invent or silently substitute a reference.

5 YAML examples

5.1 One file with multiple cases

A top-level `cases` list lets one YAML file define several named runs. The CLI executes them in order:

```
cases:
- name: normal_TE
  run: {optics: true, thermal: false}
  simulation:
    wavelength: {min: 8.0, max: 13.0, n: 201}
    angles: normal
    polarization: TE
  # geometry, structure, materials, and data follow
- name: directional_TM
  run: {optics: true, thermal: false}
  simulation:
    wavelength: {min: 8.0, max: 13.0, n: 201}
    angles: specific
    polar_angle_deg: 35.0
```

```
azimuth_angle_deg: 90.0
polarization: TM
```

YAML anchors may hold shared sections. Validation E uses one such file for nine cases and references digitized text data directly; no helper script is needed.

5.2 Normal and directional emittance

```
run:
  optics: true
  thermal: false
  plots: false
  write_outputs: true

simulation:
  wavelength: {min: 8.0, max: 13.0, n: 300}
  angles: specific
  polar_angle_deg: 35.0
  azimuth_angle_deg: 90.0
  polarization: TM
  s4_modes: 80
```

For normal incidence, set `angles: normal`; the requested TE, TM, or both are still computed explicitly.

5.3 Hemispherical emittance

```
simulation:
  wavelength: {min: 3.0, max: 30.0, n: 1000}
  angles: hemispherical
  polarization: unpolarized
  hemisphere_theta_points: 8
  hemisphere_azimuth_points: 12
  s4_modes: 100
```

A live S4 thermal case requires `angles: hemispherical`: the energy balance integrates emissivity over the hemisphere, so a single normal-incidence spectrum does not define it. Convergence must be checked by increasing the polar-angle count, azimuth count, and S4 Fourier basis size independently.

That requirement is what makes a live thermal run expensive. The solver cost scales as $n_\lambda \times n_\theta \times n_\phi \times n_{\text{pol}}$: the listing above is 1000 wavelengths over $8 \times 12 = 96$ quadrature directions, plus one zero-weight normal probe, and an unpolarized case evaluates both TE and TM at every one of those points. Changing a convection coefficient or an ambient temperature does not change any of it.

The practical workflow is therefore to run the optics once, write the reduced spectrum, and re-drive the thermal stage from that file with `run.optics_results`, which costs no S4 time. Validations A, B, and C are built this way. Validation C keeps the split explicit: four `optics_*.yaml` cases call S4 and write `data/optics/*.txt`, and two `cell_*.yaml` cases consume those spectra to solve the energy balance.

To chain the two runs without any intermediate step, set `run.optics_export` in the optics case to a fixed path and point `run.optics_results` at that same path in the thermal case. These are

the write and read halves of one five-column format (`lambda_um R T emit abs_si`). The export is written even when `run.write_outputs` is false, because it is requested by explicit path. Note that `optics.csv` cannot be reused here: it is comma-separated with a text header, which the resume reader rejects, and it lands in a timestamped folder that a later case cannot name in advance. Case A.1 in `validations/` is the minimal two-command example.

A resumed spectrum retains no directional information. Driving the thermal stage from a normal-incidence file with `run.optics_results_angles: normal` is an explicit angle-independent approximation, not a hemispherical calculation, and must be reported as such.

5.4 Layer stack and materials

```
geometry:
  source: s4
  shape: cylinder
  photonic_material: sio2
  lattice: {type: hexagonal, x: 10.608811, y: 6.125}
  cylinder: {radius: 1.75, height: 2.25}

structure:
- {material: sio2, thickness: 500.0}
- {material: silicon, thickness: 500.0}
- {material: gold, thickness: 0.08}
- {material: vacuum, thickness: 0.0, terminal: true}

materials:
  sio2: PalikKitamura_SiO2
  silicon: Akerboom_Si_lossless
  gold: RII_Olmon_2012_ev_Au
```

Exactly one terminal layer is required, it must be last, and its thickness must be zero because S4 treats it as semi-infinite.

5.5 Thermal case and temperature reduction

```
thermal:
  ambient_temperature: 300.0
  # Calibrated effective total coefficient; the paper states 6.0.
  convection_coefficient: 12.54
  absorbed_solar_power: 808.0
  reference_temperature: 360.0
  cooling_temperature: {min: 260.0, max: 380.0, n: 121}
  equilibrium: auto
```

For a digitized table whose first column is wavelength and later columns are different measured emittances, set `run.optics_results_emittance_column` to the desired zero-based column. Column zero is reserved for wavelength.

6 Outputs and reproducibility

MATLAB-style output compatibility has been removed. New runs write:

- `optics.csv`: selected directional or hemispherical spectrum;
- `optics_directional.csv`: every S4 wavelength, polar angle, azimuth, polarization, angular weight, R, T, A, A_{Si} ;
- `iv.csv`, `power.csv`, or `cooling_power.csv`;
- `run.json`: full resolved configuration, Git revision, dirty flag, interpreter/platform, S4 binary hash, input paths and SHA-256 hashes, and scalar results;
- figures when `run.plots: true`.

Set `run.write_outputs: false` for tests or programmatic validation. Historical MATLAB files remain only as read-only parity fixtures and under the gitignored `archive/`; they are never emitted by the active package.

7 Validation evidence

Case	Evidence	Conclusion
A	Published reduced spectra; full YAML thermal/PV path	Conditional thermal/PV regression. No live S4 or angular TE/TM validation; reflected-power discrepancies reach approximately 10%.
B	Digitized measured spectra and published curves	Figure 4d tests the cooling balance. Figure 5d is a digitized comparison, not an independent module calculation.
C	Four YAML-defined S4 cases; Zhao et al. grating	Normal unpolarized 8–13 μm emittance is 0.938 versus approximately 0.90; $T_{\text{eq}} = 337.82$ K versus 337.5 K. Solar absorptance is prescribed and normal optics are used as an angle-independent thermal approximation.
E	One YAML; S4 spectra and digitized Akerboom et al. Figures 3a, 5a, 5b	The 7.5–16 μm S4 RMSE is 0.025–0.030, but the silica cases have 2–16 μm RMSE 0.174 because the finite stack differs from the paper’s semi-infinite-silica absorption convention. The paper’s $h_c = 6.0$ does not reproduce Figure 5b. For the paper’s zero-emitter case it gives 434.67 K, whereas the paper reports 366.5 K; that value requires $h = 12.15 \text{ W m}^{-2} \text{ K}^{-1}$. Separate YAML cases retain the failed paper input and a joint fit $h = 12.54 \text{ W m}^{-2} \text{ K}^{-1}$, which gives 359.7/340.1/337.5 K versus 360.0/339.0/336.0 K. The latter is calibrated reproduction, not convection validation.

Validation E has not received a fresh Fourier-mode convergence sweep. Its 60-mode patterned result must not be labelled a converged reference solution.

8 Required verification for new cases

For every new publication or device case:

1. place all geometry, materials, directions, polarization, wavelength, and numerical settings in YAML;
2. record source data and reject unavailable parameters rather than inventing them;
3. verify $R + T + A$, passivity, material wavelength coverage, and layer-resolved absorption;
4. demonstrate wavelength, Fourier-mode, polar-angle, and azimuth convergence as applicable;
5. distinguish unit tests, MATLAB parity, independent optical validation, literature reproduction, digitized comparison, and calibrated fit;
6. compare published and calculated values with absolute or relative errors and retain unresolved failures.

References

- [1] V. Liu and S. Fan, “S4: A free electromagnetic solver for layered periodic structures,” <https://web.stanford.edu/group/fan/S4/>.
- [2] B. Zhao et al., “Radiative cooling of solar cells with micro-grating photonic cooler,” *Renewable Energy* 191, 662–668 (2022), <https://doi.org/10.1016/j.renene.2022.04.063>.
- [3] E. Akerboom et al., “Passive Radiative Cooling of Silicon Solar Modules with Photonic Silica Microcylinders,” *ACS Photonics* 9, 3831–3840 (2022), <https://doi.org/10.1021/acsp Photonics.2c01389>.
- [4] R. L. Olmon et al., “Optical dielectric function of gold,” *Physical Review B* 86, 235147 (2012), <https://doi.org/10.1103/PhysRevB.86.235147>.
- [5] M. N. Polyanskiy, “Refractiveindex.info database of optical constants,” *Scientific Data* 11, 94 (2024), <https://doi.org/10.1038/s41597-023-02898-2>.